Student Project

Learning Temporal Codes for Point-Process JSCC

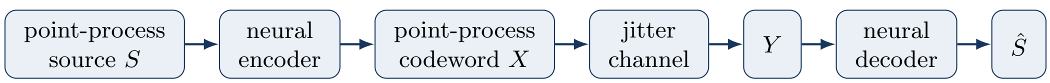
Neural joint source–channel coding with spiking neural networks over timing-jitter channels

---

Goal

Study whether a neural joint source–channel coding (JSCC) system implemented with spiking neural networks (SNNs) can **discover useful temporal codes for point processes** transmitted over a timing-jitter channel.

The basic pipeline is



with Gaussian timing jitter

$$Y_i = X_i + Z_i, \qquad Z_i \sim \mathcal{N}(0, \sigma_J^2). \tag{1}$$

The encoder should preserve the **same number of events**, so any gain must come from **how event times are rearranged**, rather than from repetition.

System Constraints and Evaluation

For a source point-process block S observed over duration T_s, the encoder produces a channel-input point process X over duration T_c.

Rate

Define the JSCC rate as

$$R = \frac{T_s}{T_c}. \tag{2}$$

In all initial experiments, choose

$$\boxed{R = 1}, \tag{3}$$

so the source and channel blocks have the same duration.

Input Cost

Use transmitted event count, or equivalently firing rate, as the channel-input cost:

$$c(X) = \frac{N_X}{T_c}, \tag{4}$$

where N_X is the number of transmitted events. In the experiments, impose the stronger sample-wise constraint

$$\boxed{N_X = N_S}, \tag{5}$$

where N_S is the number of source events. Thus, the encoder cannot gain by adding repetitions or extra spikes.

Distortion

For

$$S = \{T_1, \ldots, T_N\}, \qquad \hat{S} = \{\hat{T}_1, \ldots, \hat{T}_N\}, \tag{6}$$

define the per-block timing distortion as

$$d(S, \hat{S}) = \frac{1}{N} \sum_{i=1}^{N} \left(T_i - \hat{T}_i\right)^2. \tag{7}$$

The reported average distortion is

$$D = \mathbb{E}\left[\frac{1}{N} \sum_{i=1}^{N} \left(T_i - \hat{T}_i\right)^2\right]. \tag{8}$$

Controlled comparison. All experiments compare encoders at the same rate $R = 1$, with the same transmitted event count $N_X = N_S$, and over the same jitter channel. Therefore, any learned JSCC gain must come from **rearranging the temporal geometry of the events**.

Step 1 — Poisson source

Start with a homogeneous Poisson point process and train the neural JSCC system to reconstruct the source point process.

Compare against uncoded transmission:

$$X = S. \tag{9}$$

Remark. The Poisson process is the most "random" case considered here. Therefore, the learned encoder may simply converge to uncoded transmission.

Step 2 — Renewal source

Replace the Poisson source by a more predictable renewal process. For example, let the inter-event intervals satisfy

$$W_i = T_i - T_{i-1} \sim \Gamma\left(k, \frac{\mu}{k}\right). \tag{10}$$

Keep the mean interval μ fixed and sweep the shape parameter

$$k \in \{1, 2, 4, 8\}. \tag{11}$$

Here, $k = 1$ is Poisson-like, while larger k gives increasingly regular event timing.

Compare three systems:

uncoded vs. **learned decoder only** vs. **learned JSCC encoder + decoder**

Study both reconstruction distortion and the learned mapping of inter-event gaps,

$$g_i = T_{i+1} - T_i \qquad \longrightarrow \qquad g_i^{(X)} = X_{i+1} - X_i. \tag{12}$$

Main question for Step 2. Does increasing temporal predictability create a genuine learned JSCC gain, and what temporal transformation does the encoder learn?

Step 3 — Minimal two-event model

Once a nontrivial temporal code is observed, reduce the problem to the smallest possible model:

$$S = \{T_1, T_2\}. \tag{13}$$

Equivalently, study the single source variable

$$G = T_2 - T_1. \tag{14}$$

Train a small neural encoder–jitter–decoder system and visualize directly

$$G \longrightarrow G_X \longrightarrow G_Y \longrightarrow \hat{G}. \tag{15}$$

Vary:

- the source distribution of G;
- jitter variance;
- encoder displacement constraint;
- distortion function.

The purpose is not mainly performance. It is to **understand the mechanism learned in the larger renewal experiment, if such a mechanism exists**.

Main research question

Can neural JSCC discover that point-process information should be represented not only by individual event times, but also by the relative temporal geometry between events?

Optimization issue

Be careful about possible optimization issues.